

Procedural Icicle Project - Report Structure

1. Title / Project Overview

Title suggestion:

Procedural Icicle Scene in OpenGL

Shortly explain:

- You built a procedural icy ceiling scene.
- Icicles are generated in code, not modeled manually.
- The scene uses OpenGL, shaders, procedural mesh generation, normals, lighting, alpha blending, and a textured background.

2. Motivation

Explain why you chose this project:

I chose procedural icicle generation because it combines mesh generation, randomness, lighting, normals, and scene composition. The goal was to create a visually recognizable ice cave scene while learning low-level OpenGL concepts.

3. Problem Statement

Example:

The goal of the project was to generate many icicles procedurally within the dimensions of a roof plane, render them with lighting, and compose them with a background image to create an ice cave scene.

Mention what you wanted to achieve visually:

- Icicles attached to a roof
- Random variation in height, radius, shape, and lean
- Non-overlapping placement
- Lighting that makes the geometry readable

4. Implementation Details

This should be the largest section.

Suggested subsections:

OpenGL Setup

Explain:

- window/context using `glfwGLFW`
- VAO, VBO, EBO
- indexed drawing
- depth testing
- blending

Procedural Geometry

Explain your vertex format:

Each vertex stores position and normal:

x, y, z, nx, ny, nz

Explain how one icicle is generated:

- one tip vertex

- ring of vertices at roof height
- triangle fan from tip to ring
- indices connect the mesh

Mention your parameters:

- segment count
- min/max radius
- min/max tip height
- roof width/depth

Random Placement

Explain:

- random (x, z) positions are sampled inside the roof dimensions
- positions are rejected if too close to existing icicles
- this prevents heavy overlap

Normals and Lighting

Explain:

- normals are needed for lighting
- diffuse lighting uses dot product between normal and light direction
- specular/rim/alpha if currently in your shader

Example phrasing:

The fragment shader uses the interpolated normal to calculate how much each surface faces the light. This makes the icicles appear three-dimensional instead of flat.

Background Texture

Explain:

- iceCave.jpg is rendered as a fullscreen quad
- it uses a separate shader
- depth testing is disabled while drawing the background
- depth testing is enabled again before drawing icicles

5. Settings

=====

List important constants:

Icicle count: 6000

Segments per icicle: 24

Radius range: 0.05 to 0.20

Tip height range: -0.8 to -0.1

Roof size: 7 x 7

Also mention controls:

W/S: camera movement

1/2: wireframe/fill mode

ESC: close window

6. Results

=====

Add 1-3 screenshots in the report, and also put screenshots in a separate folder as required.

Describe what works:

- many procedural icicles
- varying shapes
- lighting visible on surfaces

- background creates scene context

7. Limitations / Future Improvements

=====

Good things to mention:

- no hardware instancing yet
- all icicle geometry is generated on CPU
- normals are approximate
- background is a flat image, not a 3D environment

Possible future improvements:

- OpenGL instancing
- better normal calculation
- child icicles on segments
- post-processing blur
- cubemap or skybox
- performance profiling

Be honest. That is better than overselling.

8. External Resources

=====

List anything you used:

- ituGL exercise framework
- GLM
- stb_image
- OpenGL documentation / LearnOpenGL if used
- StackOverflow snippet if still relevant
- source of iceCave.jpg
- any AI assistance if your course expects disclosure

Final Checklist

=====

Before hand-in, make sure you include:

- Source code
- Shader files
- Texture files
- Report, 4-8 pages
- Screenshot folder
- Build/run instructions if needed
- External resource list

The official deadline in the formal requirements PDF is Friday, May 22, 2026 at 14:00.